Capstone Deliverable 2: Architecture Diagram & Requirements

Owner: WalquerX Reviewers: Wijnaldum, Toto Status: Ready for team submission Program: Bounded Payments Repository: https://github.com/ToXMon/bounded-payments

This is the compiled group submission. It is self-contained: the requirements, the diagrams, and the matrix are in this file. The canonical working copies live in `docs/requirements.md`, `docs/architecture.md`, and `docs/adr/`.

1. Problem statement

People want software to make routine payments for them. They do not want to approve every payment by hand, and they do not want to grant unrestricted access to their money. Manual approval removes most of the value of delegation. Full wallet access turns one error or one compromised key into a large loss.

Bounded Payments is a Solana program for one recurring transfer. It holds funds in a vault and moves them only when a submitted payment satisfies rules the owner fixed in advance. The owner sets the recipient, amount, interval, number of payments, expiry, and one authorized caller key. After that the owner is out of the loop, and so is anyone holding a key that is not the authorized caller key.

Concrete scenario: a person pays a fixed monthly allowance to one recipient. An agent key submits the payment each month. If the agent key is stolen, the next due payment is the immediate exposure. Without an off-chain response, the key can trigger every remaining payment until the plan expires or reaches its limit. The thief cannot change the amount, redirect the funds, or exceed the plan limit.

2. MVP use cases

The rubric allows one to three use cases and requires **1 use case = 1 atomic state transition = 1 Anchor instruction handler**. Stage 1 has two.

Use case	State transition	Handler	Signer	Pass condition
UC-1 Create a payment plan	One payment plan PDA exists with the payment rules, and its vault exists	`create_plan`	Owner	Valid input creates both accounts. Any invalid input creates nothing.
UC-2 Execute a due transfer	Vault balance falls, recipient balance rises, `paid_total` and `next_due_at` update, or nothing changes	`execute_transfer`	Authorized caller	A due transfer succeeds. An early, expired, excessive, redirected, or unauthorized transfer changes nothing.

Funding the vault is not a program use case. Any wallet or exchange sends devnet USDC to the vault with a plain SPL Token transfer, and the Token Program enforces the mint. Funding grants no authority. The red-team recorded the old `deposit` handler as a handler that changed no program state, and the team removed it (finding F5, ADR 0002).

3. Actors

Actor categories carry forward from the team's LOI phase. Signers are marked.

LOI category	Actor	Signs	Can	Cannot
Direct actor	Owner	`create_plan`	Define all payment rules. Choose the authorized caller. Pay rent.	Change a plan after creation.
Direct actor	Authorized caller (agent key)	`execute_transfer`	Submit a due transfer. Decide when to submit.	Change rules, choose the amount or recipient, withdraw the vault.
Direct actor	Funder (any wallet or exchange)	SPL Token transfer, not a program handler	Add USDC to the vault.	Change rules or gain any authority.
Beneficiary	Recipient	Does not sign	Receive USDC.	Trigger a payment.
Administrator	Owner, for his own plan only	`create_plan`	Set the rules once.	Administer another owner's plan. There is no protocol-level administrator in Stage 1.
Stakeholder	Team	No handler	Hold the program upgrade authority on devnet.	Move funds through current Stage 1 handlers. Upgrade authority can change program behavior.
Stakeholder	Circle (USDC issuer)	No handler	Freeze any USDC token account, including the vault.	Move plan funds.
Trigger	Agent or scheduler	Does not sign	Cause an authorized caller to submit an instruction.	Bypass any on-chain check.

Outside the trust boundary: the agent or scheduler process and the web UI. They can submit transactions. They cannot change what the program verifies.

4. Protocol POC requirements

Each requirement has one action and one testable condition. Stage 1 is REQ-01 to REQ-30: REQ-01 to REQ-19 and REQ-30 for `create_plan`, REQ-20 to REQ-29 for `execute_transfer`. REQ-30 is numbered last so the `execute_transfer` range keeps its ids; it belongs to `create_plan`.

`create_plan`

Signer: the owner, who also pays rent for the plan and the vault.

Inputs: `plan_id: u64`, `amount: u64`, `interval: i64`, `payment_count: u64`, `first_due_at: Option<i64>`, `authorized_caller: Pubkey`, recipient token account.

REQ	The program must	Test
REQ-01	Create one payment plan account owned by this program at the canonical PDA from seeds `["plan", owner, plan_id]`.	Read plan. Wrong program owner, noncanonical PDA, or bump fails when the plan is used.
REQ-02	Store the owner.	Read plan.
REQ-03	Reject a mint that is not the devnet USDC mint.	Other mint fails.
REQ-04	Store the recipient token account address and its token authority.	Read both values from the plan.
REQ-05	Store the amount.	Read plan.
REQ-06	Store the interval in seconds.	Read plan.
REQ-07	Compute and store `total_limit = amount × payment_count`.	10 × 3 stores 30.
REQ-08	Compute and store `expiry = next_due_at + payment_count × interval`.	Due 100, interval 10, count 3 stores 130. No due date given, `now` 100, interval 10, count 3 stores 130.
REQ-09	Store the authorized caller.	Read plan.
REQ-10	Set `next_due_at` to `first_due_at`, or to `now` when the owner gives no value.	Both cases.
REQ-11	Set `paid_total` to 0.	Read plan.
REQ-12	Create the canonical vault ATA with the payment plan PDA as authority, using the expected System, Associated Token, and Token program IDs.	Vault address and authority match. An unexpected program ID fails.
REQ-13	Reject an amount of 0.	Fails.
REQ-14	Reject an interval of 0 or less.	0 and −1 fail.
REQ-15	Reject a payment count of 0.	Fails.
REQ-16	Reject a given `first_due_at` earlier than `now`.	Past value fails.
REQ-17	Reject a recipient token account unless it is a TokenInterface account with the devnet USDC mint.	Wrong account type or mint fails.
REQ-18	Reject the zero key as the authorized caller.	Fails.
REQ-19	Reject any arithmetic overflow in the computed fields.	Very large count or interval fails on checked math, not a panic.
REQ-30	Reject `create_plan` when the payment plan PDA for these seeds already exists.	Same owner and `plan_id` twice fails. The first plan is unchanged.

`execute_transfer`

Signer: the stored authorized caller.

REQ	The program must	Test
REQ-20	Accept a transfer request only from the stored authorized caller.	Wrong key fails.
REQ-21	Reject a request before the next due time.	`now < next_due_at` fails.
REQ-22	Reject a request after the plan expiry time.	`now > expiry` fails.
REQ-23	Reject a transfer if checked `paid_total + amount` exceeds `total_limit`.	Over-limit addition fails. If the addition overflows, REQ-28 fails it.
REQ-24	Reject a recipient token account unless its address and token authority match the values stored in the plan.	A different account or changed token authority fails.
REQ-25	Transfer the stored amount from the canonical vault ATA to the stored recipient with `transfer_checked`, using the expected Token Program ID and the plan PDA as signer.	Balances change by the amount. Wrong vault, authority, or Token Program ID and insufficient funds fail.
REQ-26	Add the transfer amount to the paid total.	Read plan.
REQ-27	Set `next_due_at` to the first scheduled time after `now`: `k = (now − next_due_at) / interval + 1`, integer division, then `next_due_at = next_due_at + k × interval`.	Due 100, interval 10, `now` 125 stores 130. `now` 100 exactly stores 110.
REQ-28	Reject any arithmetic overflow in the update.	Fails on checked math.
REQ-29	Preserve all balances and plan state when any check or transfer fails.	After each failure above, balances and plan are unchanged.

Funding the vault is not a program requirement. Any wallet or exchange sends devnet USDC to the vault with a plain SPL Token transfer, and the Token Program enforces the mint. Funding grants no authority (ADR 0002).

Required negative tests

One per rejection requirement, as `AGENTS.md` requires.

Test	REQ
Wrong mint.	REQ-03
Recipient account is not a TokenInterface account or has the wrong mint.	REQ-17
Payment plan account owned by another program.	REQ-01
Noncanonical payment plan PDA or bump when the plan is used.	REQ-01
Unexpected System, Associated Token, or Token program ID during `create_plan`.	REQ-12
Same owner and `plan_id` twice.	REQ-30
Amount of 0.	REQ-13
Interval of 0, and interval of −1.	REQ-14
Payment count of 0.	REQ-15
`first_due_at` in the past.	REQ-16
Zero key as authorized caller.	REQ-18
Overflow at creation.	REQ-19
Wrong authorized caller.	REQ-20
Transfer before the next due time.	REQ-21
Transfer after expiry.	REQ-22
Transfer above the remaining limit.	REQ-23
`paid_total + amount` overflows.	REQ-28
Wrong recipient token account address.	REQ-24
Recipient token authority changed after plan creation.	REQ-24
Vault with the wrong ATA address or authority.	REQ-25
Unexpected Token Program ID.	REQ-25
Vault with insufficient funds.	REQ-25
Overflow in the update.	REQ-28

No requirement rejects a different owner. The owner is a PDA seed, so a second owner with the same `plan_id` creates a second plan, and that is correct behavior. The rejection case is the same owner and `plan_id` twice (REQ-30).

Stage 2, provisional and outside the POC

REQ-40 to REQ-46 cover the approved purchase. They stay provisional until Stage 1 passes its tests.

- **REQ-40:** The owner must sign each change to the approved item list.
- **REQ-41:** The client must create a canonical hash for each approved purchase intent.
- **REQ-42:** The program must reject a purchase intent that the payment plan does not approve.
- **REQ-43:** The program must reject a purchase amount above the plan payment limit.
- **REQ-44:** The program must transfer tokens only to the merchant stored for the approved intent.
- **REQ-45:** The program must mark a successful purchase intent as used.
- **REQ-46:** A failed purchase must preserve all balances and program state.

Provisional handlers: `approve_purchase_intent`, `execute_purchase`.

Design decisions behind the requirements

ID	Decision	Requirements
D1	`create_plan` stores one `authorized_caller` key. The owner chooses it. That key can only call `execute_transfer`.	REQ-09, REQ-18, REQ-20
D2	Anyone can fund the vault with a plain SPL Token transfer. Funding gives no authority.	none
D3	The schedule uses Unix time from the Clock sysvar, in seconds, with a fixed `interval`.	REQ-10, REQ-16, REQ-21, REQ-22, REQ-27
D4	A failed or late payment is a missed payment. After a late success, `next_due_at` becomes the first scheduled time after `now`. Maximum one payment per interval.	REQ-27
D5	The token is devnet USDC on the classic SPL Token Program, accessed through Anchor's `TokenInterface`. The mint is a program constant.	REQ-03, REQ-17
D6	`create_plan` creates the vault. The owner pays the rent for the plan and the vault.	REQ-01, REQ-12
D7	`create_plan` rejects every invalid input, because the plan cannot change after creation.	REQ-13 to REQ-18, REQ-30
D8	The payment rules are immutable.	REQ-13 to REQ-18
D9	`first_due_at` is optional. If the owner gives no value, the first payment is due now.	REQ-10
D10	The owner gives `amount`, `interval`, and `payment_count`. The program computes `total_limit = amount × payment_count` and `expiry = next_due_at + payment_count × interval`. The last payment has one full interval to execute.	REQ-07, REQ-08
D11	All arithmetic uses checked math. An overflow fails the transaction.	REQ-19, REQ-28
D12	Stage 1 has two handlers: `create_plan` and `execute_transfer`.	all

5. Overview diagram

This is the main architecture overview. Its actor arrows show handler requirement ranges, and its money paths separate plain-token funding from program-authorized transfers. The handler diagrams below map checks to individual REQ ids.

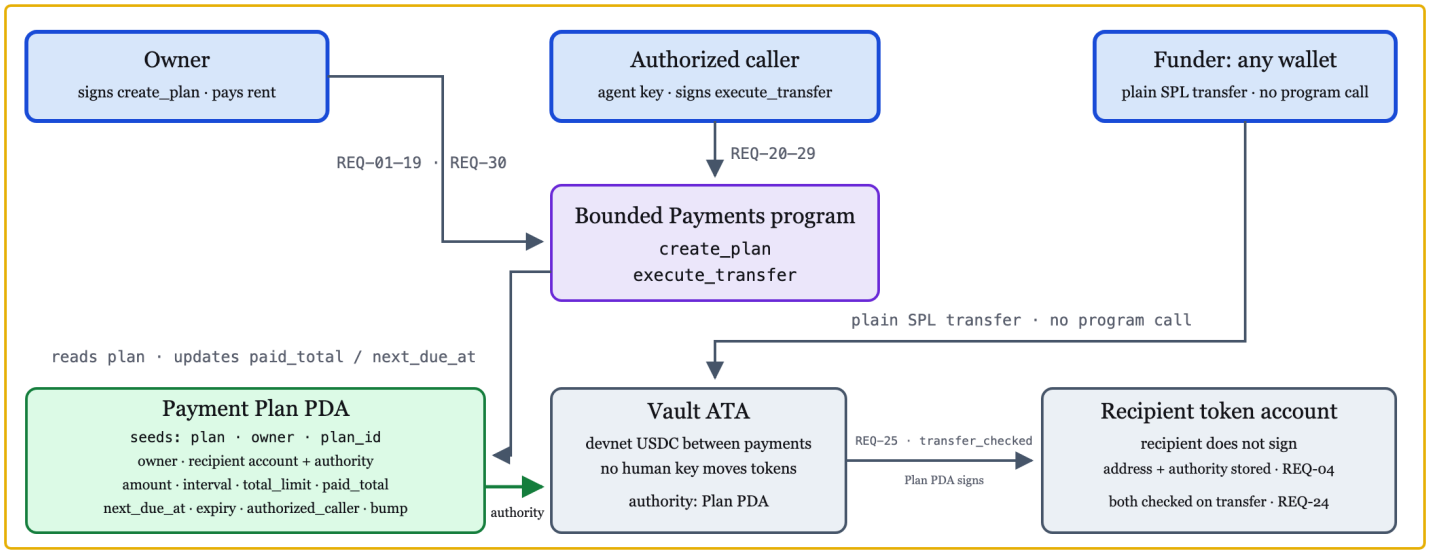


Figure 1: Main architecture overview

Vector PDF alternative: overview.svg.

What each path shows:

- **Owner → `create_plan` (REQ-01 to REQ-19, REQ-30).** The owner signs. The program validates inputs, creates the plan PDA and vault, and stores the payment rules. D2 shows the checks and writes.
- **Authorized caller → `execute_transfer` (REQ-20 to REQ-29).** The caller signs. D3 shows the ordered checks, transfer, state update, and shared failure path. D4 shows Clock and program dependencies.
- **Plan PDA → vault (REQ-12).** The PDA is the vault's token authority. No person controls that key.
- **Funder → vault.** Any wallet can send a plain SPL Token transfer. It does not call this program.
- **Vault → recipient (REQ-25).** `transfer_checked` moves the stored amount, signed by the Plan PDA. REQ-24 checks the recipient account address and token authority before the transfer.
- **Program → plan (REQ-26, REQ-27).** After a successful transfer, the program updates `paid_total` and `next_due_at`. REQ-29 preserves state and balances on failure.

Funding is a separate plain SPL Token transfer from any wallet or exchange. The Token Program enforces the mint. Funding grants no authority.

The vault's authority is a program-derived address that no person controls. The team holds the upgrade authority, so it can change program behavior unless that authority is later burned or governed.

6. Flow diagrams

One flow per handler. Every decision shows both branches, and every failure path reaches one error node.

`create_plan`

The owner signs. The diagram groups input checks for readability; the requirements matrix lists each check and rejection case. Any failed check creates no account.

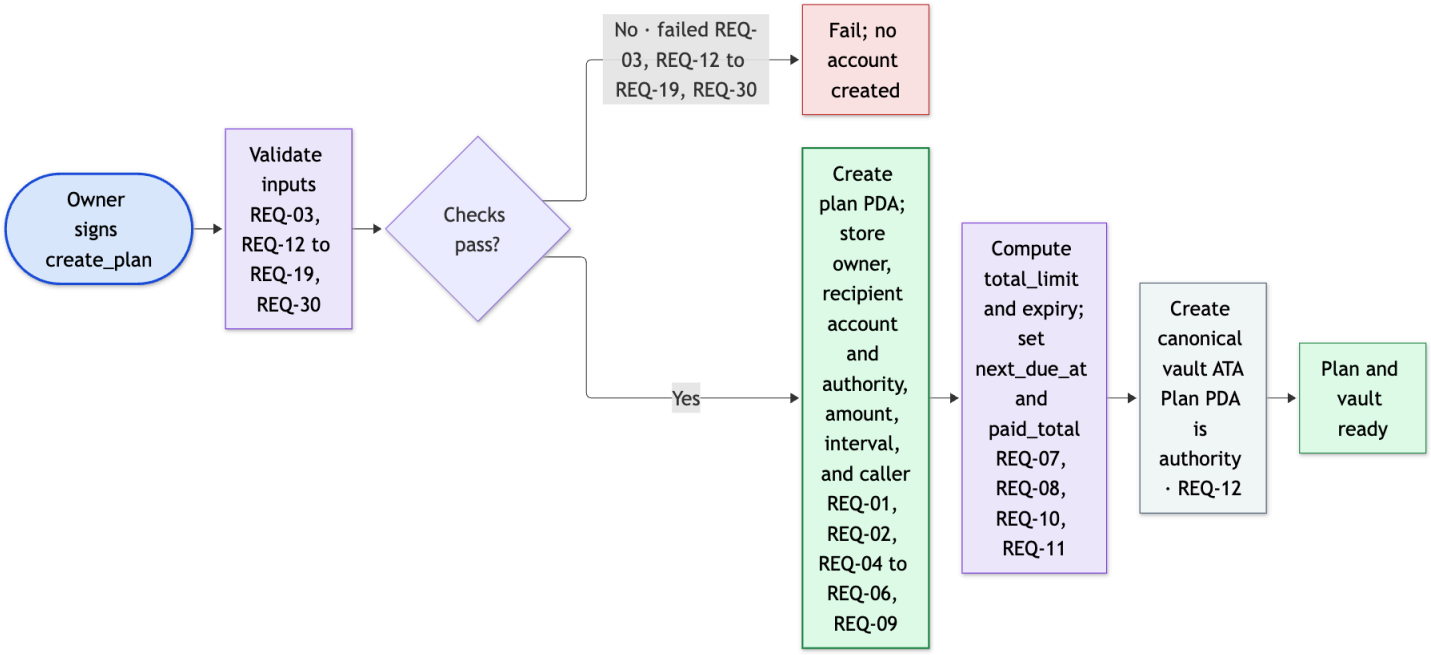


Figure 2: create_plan decision flow

Vector PDF alternative: create-plan.svg.

The program then creates the plan PDA, stores the payment rules, computes `total_limit` and `expiry`, sets `paid_total` to 0 and `next_due_at`, and creates the vault with the plan PDA as authority (REQ-02, REQ-04 to REQ-12).

`execute_transfer`

The authorized caller signs. Every decision, including the Token Program CPI result, has a failure path into the shared error node.

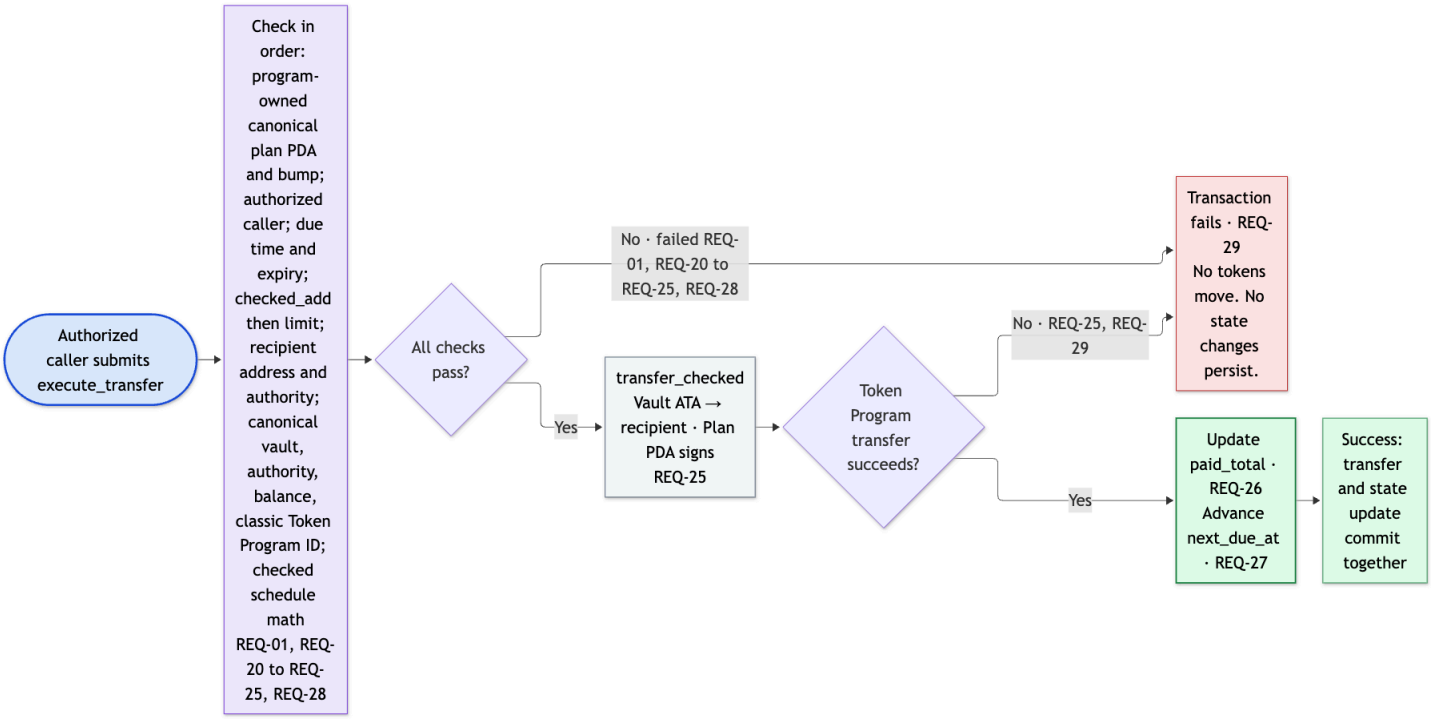


Figure 3: execute_transfer decision flow

Vector PDF alternative: execute-transfer.svg.

If the Token Program rejects `transfer_checked`, the transaction fails. Otherwise the CPI moves the amount from the vault to the recipient, signed by the plan PDA (REQ-25), `paid_total` grows (REQ-26), and `next_due_at` becomes the first scheduled time after `now` (REQ-27). Any failure leaves every balance and every plan field unchanged (REQ-29).

A late call

Plan with `amount` 10, `interval` 10, `payment_count` 3: payments due at 100, 110, and 120, `expiry` 130.

Call lands at	Result	`next_due_at` after
100	One payment	110
125	One payment. The 110 and 120 slots are gone. Late does not stack.	130
131	Rejected. Past `expiry`.	130, unchanged

`k = (now − next_due_at) / interval + 1`, integer division, then `next_due_at = next_due_at + k × interval`.

7. End-to-end overview

The off-chain actors around the program, and the external programs and sysvars it calls.

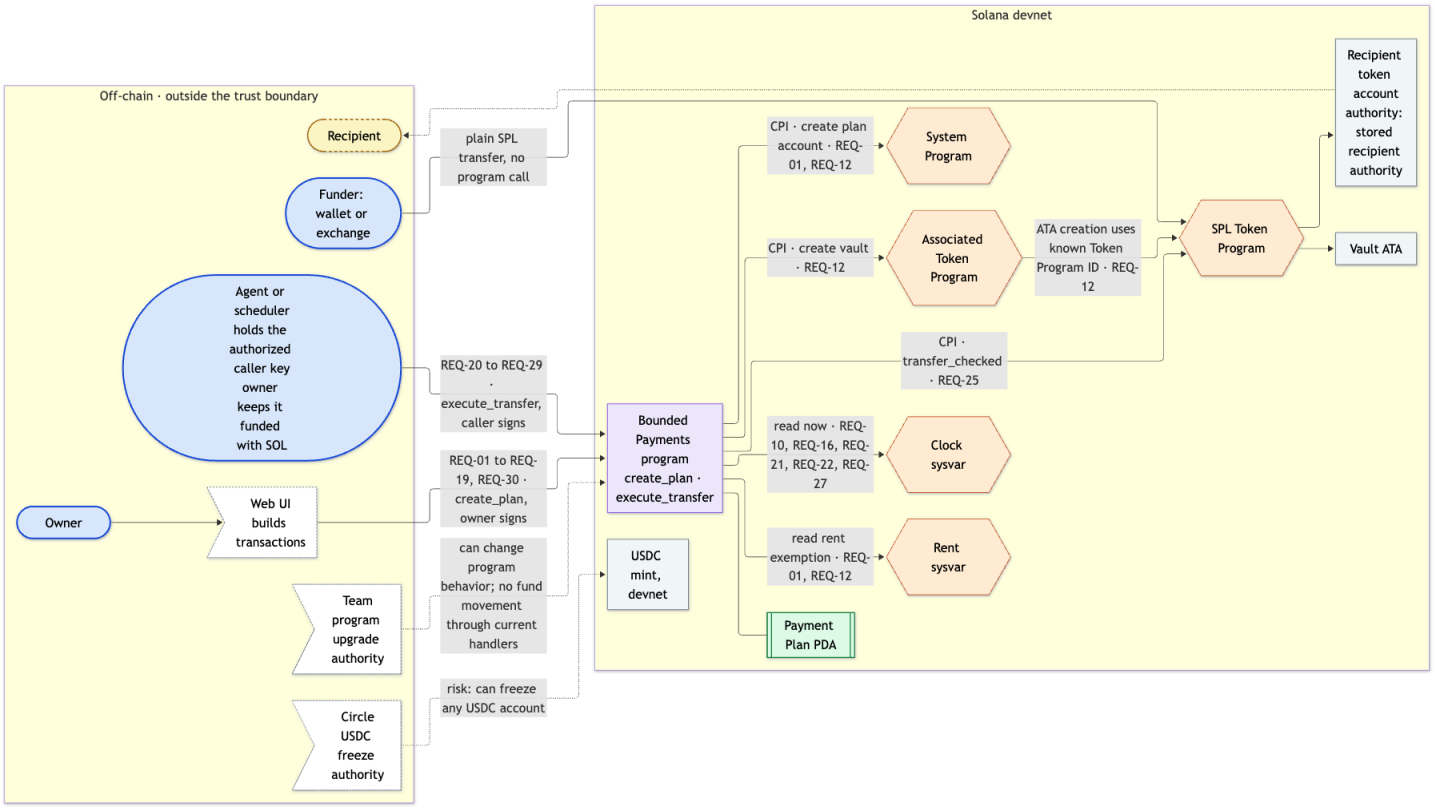


Figure 4: End-to-end architecture diagram

Vector PDF alternative: end-to-end.svg.

The owner builds the `create_plan` transaction in the web UI and signs it. The agent or scheduler holds the authorized caller key and submits `execute_transfer` when a payment is due. The funder sends devnet USDC to the vault with a plain SPL Token transfer. Circle, the USDC issuer, can freeze any USDC account, including the vault. The team holds the upgrade authority and can change program behavior. The program never executes by itself.

8. On-chain requirements matrix

Stage 1 uses one program PDA, two token accounts, one mint, and two handlers. Program IDs are listed because `AGENTS.md` requires accepting only known program IDs for CPI.

Accounts

Account	Type	Owner	Authority	Seeds or derivation	Fields
Payment plan	PDA	Bounded Payments	Program	`["plan", owner, plan_id, canonical bump]`	`owner: Pubkey, plan_id: u64, recipient_token_account: Pubkey, recipient_authority: Pubkey, amount: u64, interval: i64, total_limit: u64, paid_total: u64, next_due_at: i64, expiry: i64, authorized_caller: Pubkey, bump: u8`
Vault	Token account (ATA)	Token Program	Payment plan PDA	ATA of (plan PDA, USDC mint)	USDC balance
Recipient token account	Token account	Token Program	Stored recipient authority	Given by the owner; address and authority stored and re-checked on every transfer	USDC balance
USDC mint	Mint	Token Program	Circle	Program constant `4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU`	6 decimals, freeze authority `5EykT1...`

Anchor `init` derives the canonical bump. `execute_transfer` re-derives the vault ATA from the stored seeds and checks the bump. The recipient account must be a token account owned by the Token Program, with the pinned mint. Store its address and token authority at creation, then check both on every transfer.

Handlers

Handler	Signer	CPIs	Account constraints	REQ
`create_plan`	Owner, pays rent	System Program (create plan), Associated Token Program (create vault; uses Token Program ID)	Expected System, Associated Token, and Token program IDs; `init` plan PDA with seeds; `mint == USDC_DEVNET`; recipient is a TokenInterface account with the pinned mint; vault `init` with authority = plan PDA	REQ-01 to REQ-19, REQ-30
`execute_transfer`	Authorized caller	`TokenInterface::transfer_checked`; plan PDA signs with seeds	`caller == plan.authorized_caller`; recipient address and authority match the plan; vault == ATA(plan, mint); re-derive and check the bump; time, limit, and overflow checks in the handler	REQ-20 to REQ-29

Anchor 0.31, per `AGENTS.md`. Read `decimals` from the mint at runtime; `transfer_checked` verifies it.

External dependencies

Each handler compares each incoming program address against this list. It does not trust the account owner field alone.

Dependency	Program ID	Used by
SPL Token Program (classic)	`TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA`	`create_plan` via ATA creation, `execute_transfer`, funding
Associated Token Program	`ATokenGPvbdGVxr1b2hvZbsiqW5xWH25efTNsLJA8knL`	`create_plan`
System Program	`11111111111111111111111111111111`	`create_plan`
Clock sysvar	`SysvarC1ock11111111111111111111111111111111`	Both handlers
Rent sysvar	`SysvarRent111111111111111111111111111111111`	Both `init` constraints in `create_plan` (D6)
Circle (USDC issuer)	off-chain authority	Can freeze any USDC account, including the vault
Agent or scheduler	off-chain process	Submits `execute_transfer`
Web UI	off-chain client	Builds `create_plan`

9. Granularity self-check

Run by the team before any AI tooling, as the rubric requires. It first ran on three handlers; the red-team pass later removed `deposit`.

	Rule	`create_plan`	`execute_transfer`
1.	Atomicity	Pass. One handler. It creates the plan PDA and the vault together; both succeed or both fail.	Pass. One handler. Checks, transfer, and state update commit together.
2.	State ownership	Pass. Plan PDA is program-owned. Vault is owned by the Token Program with the plan PDA as authority. No off-chain state.	Pass. Writes the plan PDA, the vault, and the recipient token account. No off-chain state.
3.	Real signers	Pass. The owner signs.	Pass. The authorized caller signs. That key decides only when to submit.
4.	On-chain vs client	Pass. On-chain: input checks, computed fields, account creation. Client: choosing the values, building the transaction.	Pass. On-chain: caller, time, limit, and recipient checks, the transfer, the state update. Client: deciding when to submit.

Off-chain (client): the agent or scheduler, the web UI, plan value selection, notifications.

10. AI red-team record

The red-team pass ran after the self-check, against three handlers. Eight findings came back. The team accepted five and rejected three, two of those with its own alternative.

ID	Finding	Decision
F1	`interval` is `i64`, so a negative value passes a "not 0" check	Accept. REQ-14 rejects `interval ≤ 0`.
F2	`next_due_at` math can overflow `i64`	Accept. REQ-19 and REQ-28 use checked math.
F3	Nothing funds the caller key for transaction fees	Reject. The owner tops up that key. Assumption A2.
F4	Make `execute_transfer` permissionless so late calls cannot miss	Reject. The owner chooses and trusts one caller; in Stage 2 the agent's discretion is the product. REQ-20 stays.
F5	`deposit` changes no program state	Accept. Handler removed. Stage 1 is two handlers. ADR 0002.
F6	`first_due_at = now` can fail because the transaction lands later	Reject with an alternative: the owner leaves `first_due_at` empty to mean now. D9, REQ-10, REQ-16.
F7	`total_limit` can be unreachable before `expiry`	Accept with an alternative: derive the fields instead of checking them. D10, REQ-07, REQ-08.
F8	The recipient can close its account, or Circle can freeze it	Accept as known limitation L2.

11. Assumptions and limitations

ID	Assumption
A1	The program never executes by itself. Each payment needs a signed `execute_transfer`.
A2	The owner keeps the authorized caller key funded with SOL for fees.
A3	The authorized caller submits on time. A late submission makes the recipient miss that payment.

ID	Limitation	Roadmap fix
L1	Tokens left in the vault after `expiry` stay locked, and the owner's rent is not returned.	`close_plan`
L2	If the recipient closes its account, or Circle freezes the vault or the recipient account, every transfer fails.	None planned
L3	A plan cannot change after creation.	Plan updates

12. Milestones and roadmap

1. Week 1: `create_plan`, all input checks, positive and negative tests for REQ-01 to REQ-19 and REQ-30.
2. Week 2: `execute_transfer`, all checks, positive and negative tests for REQ-20 to REQ-29.
3. Stage 2 starts only after every Stage 1 test passes.

Roadmap, not in the POC: `close_plan`; Stage 2 approved purchases (REQ-40 to REQ-46); plan updates; calendar schedules; Token-2022; multiple callers and key rotation; threshold confirmation; passkey owner approval; multiple recipients; pause and resume; receipt indexer; x402 integration.

13. Resolved decisions

All four questions this section used to hold are now decided.

Question	Decision	Where
Who can act as the authorized caller?	One stored key, chosen by the owner. The zero key is rejected.	D1, REQ-09, REQ-18
Does a late transfer skip or catch up?	It skips. One payment per interval, then the first scheduled time after `now`.	D4, REQ-27
Which token does the POC use?	Devnet USDC, classic program, accessed through `TokenInterface`.	D5, ADR 0002
Can the owner change an active plan?	No. Immutability is why every invalid input is rejected at creation.	D7, D8, L3